



# Using the Populate Utility

Version 2026.1  
2026-04-20

*Using the Populate Utility*

PDF generated on 2026-04-20

InterSystems IRIS Version 2026.1

Copyright © 2026 InterSystems Corporation

All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

**InterSystems Worldwide Response Center (WRC)**

Tel: +1-617-621-0700

Tel: +44 (0) 844 854 2917

Email: [support@InterSystems.com](mailto:support@InterSystems.com)

# Table of Contents

|                                                                          |          |
|--------------------------------------------------------------------------|----------|
| <b>Using the Populate Utility</b> .....                                  | <b>1</b> |
| 1 Data Population Basics .....                                           | 1        |
| 1.1 Populate() Details .....                                             | 2        |
| 2 Default Behavior .....                                                 | 3        |
| 2.1 Literal Properties .....                                             | 3        |
| 2.2 Collection Properties .....                                          | 4        |
| 2.3 Properties That Refer to Serial Objects .....                        | 5        |
| 2.4 Properties That Refer to Persistent Objects .....                    | 5        |
| 2.5 Relationship Properties .....                                        | 5        |
| 3 Specifying the POPSPEC Parameter .....                                 | 6        |
| 3.1 Specifying the POPSPEC Parameter for Non-Collection Properties ..... | 6        |
| 3.2 Specifying the POPSPEC Parameter for List Properties .....           | 7        |
| 3.3 Specifying the POPSPEC Parameter for Array Properties .....          | 7        |
| 3.4 Specifying the POPSPEC Parameter via an SQL Table .....              | 8        |
| 4 Basing One Generated Property on Another .....                         | 8        |
| 5 How %Populate Works .....                                              | 9        |
| 6 Custom Populate Actions and the OnPopulate() Method .....              | 10       |
| 7 Alternative Approach: Creating a Utility Method .....                  | 10       |
| 7.1 Tips for Building Structure into the Data .....                      | 11       |



# Using the Populate Utility

InterSystems IRIS® data platform includes a utility, *populate utility*, for creating pseudo-random test data for persistent classes. Test data enables you to verify how your application will function when working against a large set of data.

The populate utility takes its name from its principal element — the %Populate class. Classes that inherit from %Populate contain a method called **Populate()**, which allows you to generate and save class instances containing valid data. You can also customize the behavior of the %Populate class to provide data for your needs.

Along with the %Populate class, the populate utility includes the %PopulateUtils class, which provides additional helper methods.

The Samples-Data sample (<https://github.com/interSystems/Samples-Data>) uses the populate utility. If you use that sample, InterSystems recommends that you create a dedicated namespace called SAMPLES (for example) and load the sample into that namespace. For the general process, see [Downloading Samples for Use with InterSystems IRIS](#).

## 1 Data Population Basics

To use the populate utility, do the following:

1. Modify each persistent and each serial class that you want to populate with data. Specifically, add %Populate to the end of the list of superclasses, so that the class inherits the interface methods. For example, if a class inherits directly from %Persistent, its new superclass list would be:

### Class Definition

```
Class MyApp.MyClass Extends (%Persistent,%Populate) {}
```

Do not use %Populate as a *primary* superclass; that is, do not list it as the first class in the superclass list.

2. In those classes, optionally specify the *POPSPEC* and *POPORDER* parameters of each property, to control how the populate utility generates data for those properties, if you want to generate custom data rather than the default data, which is described in the [next section](#).
3. Recompile the classes.
4. To generate the data, call the **Populate()** method of each persistent class. By default, this method generates 10 records for the class (including any serial objects that it references):

### ObjectScript

```
Do ##class(MyApp.MyClass).Populate()
```

If you prefer, you can specify the number of objects to create:

### ObjectScript

```
Do ##class(MyApp.MyClass).Populate(num)
```

where *num* is the number of objects that you want.

Do this in the same order in which you would add records manually for the classes. That is, if Class A has a property that refers to Class B, use the following table to determine which class to populate first:

| If the property in Class A has this form...                                                                                                                                                                   | And Class B inherits from... | Populate this class first...                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|----------------------------------------------|
| Any of these forms: <ul style="list-style-type: none"> <li>Property PropertyName as ClassB;</li> <li>Property PropertyName as List of ClassB;</li> <li>Property PropertyName as Array of ClassB;</li> </ul>   | %SerialObject                | ClassA (this populates ClassB automatically) |
| Any of these forms: <ul style="list-style-type: none"> <li>Property PropertyName as ClassB;</li> <li>Property PropertyName as List of ClassB;</li> <li>Property PropertyName as Array of ClassB;</li> </ul>   | %Persistent                  | ClassB                                       |
| Any of these forms: <ul style="list-style-type: none"> <li>Relationship PropertyName as ClassB [ Cardinality = one ...];</li> <li>Relationship PropertyName as ClassB [ Cardinality = parent ...];</li> </ul> | %SerialObject or %Persistent | ClassB                                       |
| Any of these forms: <ul style="list-style-type: none"> <li>Relationship PropertyName as ClassB [ Cardinality = many...];</li> <li>Relationship PropertyName as ClassB [ Cardinality = child ...];</li> </ul>  | %SerialObject or %Persistent | ClassA                                       |

Later, to remove the generated data, use either the **%DeleteExtent()** method (safe) or the **%KillExtent()** method (fast) of the persistent interface. For more information, see [Deleting Saved Objects](#).

**Tip:** In practice, it is often necessary to populate classes repeatedly, as you make changes to your code. Thus it is useful to write a method or a routine to populate classes in the correct order, as well as to remove the generated data.

## 1.1 Populate() Details

Formally, the **Populate()** class method has the following signature:

```
classmethod Populate(count As %Integer = 10,
                    verbose As %Integer = 0,
                    DeferIndices As %Integer = 1,
                    ByRef objects As %Integer = 0,
                    tune As %Integer = 1,
                    deterministic As %Integer = 0) as %Integer
```

Where:

- count* is the desired number of objects to create.
- verbose* specifies whether the method should print progress messages to the current device.

- *DeferIndices* specifies whether to sort indexes after generating the data (true) or while generating the data.
- *objects*, which is passed by reference, is an array that contains the generated objects.
- *tune* specifies whether to run **\$SYSTEM.SQL.TuneTable()** after generating the data. If this is 0, the method does not run **\$SYSTEM.SQL.TuneTable()**. If this is 1 (the default), the method runs **\$SYSTEM.SQL.TuneTable()** for this table. If this is any value higher than 1, the method runs **\$SYSTEM.SQL.TuneTable()** for this table and for any tables projected by persistent superclasses of this class.
- *deterministic* specifies whether to generate the same data each time you call the method. By default, the method generates different data each time you call it.

**Populate()** returns the number of objects actually populated:

### ObjectScript

```
Set objs = ##class(MyApp.MyClass).Populate(100)
// objs is set to the number of objects created.
// objs will be less than or equal to 100
```

In cases with defined constraints, such as a minimum or maximum length, some of the generated data may not pass validation, so that individual objects will not be saved. In these situations, **Populate()** may create fewer than the specified number of objects.

If errors prevent objects from being saved, and this occurs 1000 times sequentially with no successful saves, **Populate()** quits.

## 2 Default Behavior

This section describes how the **Populate()** method generates data, by default, for the following kinds of properties:

- [Literal properties](#)
- [Collection properties](#)
- [Properties that refer to serial objects](#)
- [Properties that refer to persistent objects](#)
- [Relationship properties](#)

The **Populate()** method ignores stream properties.

### 2.1 Literal Properties

This section describes how the **Populate()** method, by default, generates data for properties of the forms:

```
Property PropertyName as Type;
Property PropertyName;
```

Where *Type* is a datatype class.

For these properties, the **Populate()** method first looks at the name. Some property names are handled specially, as follows:

| If the property name is any case variation of the following | Populate() invokes the following method to generate data for it |
|-------------------------------------------------------------|-----------------------------------------------------------------|
| NAME                                                        | Name()                                                          |
| SSN                                                         | SSN()                                                           |
| COMPANY                                                     | Company()                                                       |
| TITLE                                                       | Title()                                                         |
| PHONE                                                       | USPhone()                                                       |
| CITY                                                        | City()                                                          |
| STREET                                                      | Street()                                                        |
| ZIP                                                         | USZip()                                                         |
| MISSION                                                     | Mission()                                                       |
| STATE                                                       | USState()                                                       |
| COLOR                                                       | Color()                                                         |
| PRODUCT                                                     | Product()                                                       |

If the property does not have one of the preceding names, then the **Populate()** method looks at the property type and generates suitable values. For example, if the property type is %String, the **Populate()** method generates random strings (respecting the *MAXLEN* parameter of the property). For another example, if the property type is %Integer, the **Populate()** method generates random integers (respecting the *MINVAL* and *MAXVAL* parameters of the property).

If the property does not have a type, InterSystems IRIS assumes that it is a string. This means that the **Populate()** method generates random strings for its values.

### 2.1.1 Exceptions

The **Populate()** method does not generate data for a property if the property is private, is multidimensional, is calculated, or has an initial expression.

## 2.2 Collection Properties

This section describes how the **Populate()** method, by default, generates data for properties of the forms:

```
Property PropertyName as List of Classname;
Property PropertyName as Array of Classname;
```

For such properties:

- If the referenced class is a data type class, the **Populate()** method generates a list or array (as suitable) of values, using the logic described earlier for [data type classes](#).
- If the referenced class is a serial object, the **Populate()** method generates a list or array (as suitable) of serial objects, using the logic described earlier for [serial objects](#).
- If the referenced class is a persistent class, the **Populate()** method performs a random sample of the extent of the referenced class, randomly selects values from that sample, and uses those to generate a list or array (as suitable).

## 2.3 Properties That Refer to Serial Objects

This section describes how the **Populate()** method, by default, generates data for properties of the form:

```
Property PropertyName as SerialObject;
```

Where *SerialObject* is a class that inherits from %SerialObject.

For such properties:

- If the referenced class inherits from %Populate, the **Populate()** method creates an instance of the class and generates property values as described in the preceding section.
- If the referenced class does not inherit from %Populate, the **Populate()** method does not generate any values for the property.

## 2.4 Properties That Refer to Persistent Objects

This section describes how the **Populate()** method, by default, generates data for properties of the following form:

```
Property PropertyName as PersistentObject;
```

Where *PersistentObject* is a class that inherits from %Persistent.

For such properties:

- If the referenced class inherits from %Populate, the **Populate()** method performs a random sample of the extent of the referenced class and then randomly selects one value from that sample.  
Note that this means you must generate data for the referenced class first. Or create data for the class in any other way.
- If the referenced class does not inherit from %Populate, the **Populate()** method does not generate any values for the property.

For information on relationships, see the [next section](#).

## 2.5 Relationship Properties

This section describes how the **Populate()** method, by default, generates data for properties of the following form:

```
Relationship PropertyName as PersistentObject;
```

Where *PersistentObject* is a class that inherits from %Persistent.

For such properties:

- If the referenced class inherits from %Populate:
  - If the cardinality of the relationship is one or parent, then the **Populate()** method performs a random sample of the extent of the referenced class and then randomly selects one value from that sample.  
Note that this means you must generate data for the referenced class first. Or create data for the class in any other way.
  - If the cardinality of the relationship is many or children, then the **Populate()** method ignores this property because the values for this property are not stored in the extent for this class.
- If the referenced class does not inherit from %Populate, the **Populate()** method does not generate any values for the property.

## 3 Specifying the POPSPEC Parameter

For a given property in a class that extends %Populate, you can customize how the **Populate()** method generates data for that property. To do so, do the following:

- Find or create a method that returns a random, but suitable value for this property.  
The %PopulateUtils class provides a large set of such methods; see the Class Reference for details.
- Specify the *POPSPEC* parameter for this property to refer to this method. The [first subsection](#) gives the details.

The *POPSPEC* parameter provides additional options for [list](#) and [array](#) properties, discussed in later subsections.

For a literal, non-collection property, another technique is to identify an SQL table column that contains values to use for this property; then specify the *POPSPEC* parameter to refer to this property; see the [last subsection](#).

**Note:** There is also a *POPSPEC* parameter defined at the class level that controls data population for an entire class. This is an older mechanism (included for compatibility) that is replaced by the property-specific *POPSPEC* parameter. This topic does not discuss it further.

### 3.1 Specifying the POPSPEC Parameter for Non-Collection Properties

For a literal property that is not a collection, use one of the following variations:

- *POPSPEC*="MethodName ( )" — In this case, **Populate()** invokes the class method *MethodName\** ( of the %PopulateUtils class.
- *POPSPEC*=".MethodName ( )" — In this case, **Populate()** invokes the instance method *MethodName ( )* of the instance that is being generated.
- *POPSPEC*="##class (ClassName) .MethodName ( )" — In this case, **Populate()** invokes the class method *MethodName ( )* of the *ClassName* class.

For example:

#### Class Member

```
Property HomeCity As %String(POPSPEC = "City()");
```

If you need to pass a string value as an argument to the given method, double the starting and closing quotation marks around that string. For example:

#### Class Member

```
Property PName As %String(POPSPEC = "Name(\"\"F\"\")");
```

Also, you can append a string to the value returned by the specified method. For example:

#### Class Member

```
Property JrName As %String(POPSPEC = "Name()_\"\" jr.\"\" ");
```

Notice that it is necessary to double the starting and closing quotation marks around that string. It is not possible to prepend a string, because the *POPSPEC* is assumed to start with a method.

Also see [Specifying the POPSPEC Parameter via an SQL Table](#) for a different approach.

## 3.2 Specifying the POPSPEC Parameter for List Properties

For a property that is a list of literals or objects, you can use the following variation:

```
POPSPEC="basicspec:MaxNo"
```

Where

- *basicspec* is one of the basic variations shown in the preceding section. Leave *basicspec* empty if the property is a list of objects.
- *MaxNo* is the maximum number of items in the list; the default is 10.

For example:

### Class Member

```
Property MyListProp As list Of %String(POPSPEC = ".MyInstanceMethod():15");
```

You can omit *basicspec*. For example:

### Class Member

```
Property Names As list of Name(POPSPEC=":3");
```

In the following examples, there are lists of several types of data. Colors is a list of strings, Kids is a list of references to persistent objects, and Addresses is a list of embedded objects:

```
Property Colors As list of %String(POPSPEC="ValueList("",Red,Green,Blue""));
```

```
Property Kids As list of Person(POPSPEC=":5");
```

```
Property Addresses As list of Address(POPSPEC=":3");
```

To generate data for the Colors property, the **Populate()** method calls the **ValueList()** method of the PopulateUtils class. Notice that this example passes a comma-separated list as an argument to this method. For the Kids property, there is no specified method, which results in automatically generated references. For the Addresses property, the serial Address class inherits from %Populate and data is automatically populated for instances of the class.

## 3.3 Specifying the POPSPEC Parameter for Array Properties

For a property that is an array of literals or objects, you can use the following variation:

```
POPSPEC="basicspec:MaxNo:KeySpecMethod"
```

Where:

- *basicspec* is one of the basic variations shown earlier. Leave *basicspec* empty if the property is an array of objects.
- *MaxNo* is the maximum number of items in the array. The default is 10.
- *KeySpecMethod* is the specification of the method that generates values to use for the keys of the array. The default is `String()`, which means that InterSystems IRIS invokes the **String()** method of %PopulateUtils.

The following examples show arrays of several types of data and different kinds of keys:

```
Property Tix As array of %Integer(POPSPEC="Integer():20:Date()");
```

```
Property Reviews As array of Review(POPSPEC=":3:Date()");
```

```
Property Actors As array of Actor(POPSPEC=":15:Name()");
```

The Tix property has its data generated using the **Integer()** method of the PopulateUtils class; its keys are generated using the **Date()** method of the PopulateUtils class. The Reviews property has no specified method, which results in automatically generated references, and has its keys also generated using the **Date()** method. The Actors property has no specified method, which results in automatically generated references, and has its keys generated using the **Name()** method of the PopulateUtils class.

## 3.4 Specifying the POPSPEC Parameter via an SQL Table

For *POPSPEC*, rather than specifying a [method](#) that returns a random value, you can specify an SQL table name and an SQL column name to use. If you do so, then the **Populate()** method constructs a dynamic query to return the distinct column values from that column of that table. For this variation of *POPSPEC*, use the following syntax:

```
POPSPEC=":MaxNo:KeySpecMethod:SampleCount:Schema_Table:ColumnName"
```

Where:

- *MaxNo* and *KeySpecMethod* are optional and apply only to collection properties (see earlier the subsections on [lists](#) and [arrays](#)).
- *SampleCount* is the number of distinct values to retrieve from the given column, to use as a starting point. If this is larger than the number of existing distinct values in that column, then all values are possibly used.
- *Schema\_Table* is the name of the table.
- *ColumnName* is the name of the column.

For example:

### Class Member

```
Property P1 As %String(POPSPEC=":::100:Wasabi_Data.Outlet:Phone");
```

In this example, the property P1 receives a random value from a list of 100 phone numbers retrieved from the Wasabi\_Data.Outlet table.

# 4 Basing One Generated Property on Another

In some cases, the set of suitable value for one property (A) might depend upon the existing value of another property (B). In such a case:

- Create an instance method to generate values for property A. In this method, use instance variables to obtain the value of property B (and any other properties that should be considered). For example:

### Class Member

```
Method MyMethod() As %String
{
    if (i%MyBooleanProperty) {
        quit "abc"
    } else {
        quit "def"
    }
}
```

For more information on instance variables, see [i%PropertyName](#).

Use this method in the *POPSPEC* parameter of the applicable property. See [Specifying the POPSPEC Parameter](#).

- Specify the *POPORDER* parameter of any properties that must be populated in a specific order. This parameter should equal an integer. InterSystems IRIS populates properties with lower values of *POPORDER* before properties with higher values of *POPORDER*. For example:

```
Property Name As %String(POPORDER = 2, POPSPEC = ".MyNameMethod()");
Property Gender As %String(POPORDER = 1, VALUelist = ",1,2");
```

## 5 How %Populate Works

This section describes how %Populate works internally. The %Populate class contains two method generators: **Populate()** and **PopulateSerial()**. Each persistent or serial class inheriting from %Populate has one or the other of these two methods included in it (as appropriate).

We will describe only the **Populate** method here. The **Populate()** method is a loop, which is repeated for each of the requested number of objects.

Inside the loop, the code:

1. Creates a new object
2. Sets values for its properties
3. Saves and closes the object

A simple property with no overriding *POPSPEC* parameter has a value generated using code with the form:

### ObjectScript

```
Set obj.Description = ##class(%PopulateUtils).String(50)
```

While using a library method from %PopulateUtils via a Name:Name() specification would generate:

### ObjectScript

```
Set obj.Name = ##class(%PopulateUtils).Name()
```

An embedded Home property might create code like:

### ObjectScript

```
Do obj.HomeSetObject(obj.Home.PopulateSerial())
```

The generator loops through all the properties of the class, and creates code for some of the properties, as follows:

1. It checks if the property is private, is calculated, is multidimensional, or has an initial expression. If any of these are true, the generator exits.
2. If the property has a *POPSPEC* override, the generator uses that and then exits.
3. If the property is a reference, on the first time through the loop, the generator builds a list of random IDs, takes one from the list, and then exits. For the subsequent passes, the generator simply takes an ID from the list and then exits.
4. If the property name is one of the specially handled names, the generator then uses the corresponding library method and then exits.
5. If the generator can generate code based on the property type, it does so and then exits.
6. Otherwise, the generator sets the property to an empty string.

Refer to the %PopulateUtils class for a list of available methods.

## 6 Custom Populate Actions and the OnPopulate() Method

For additional control over the generated data, you can define an **OnPopulate()** method. If an **OnPopulate()** method is defined, then the **Populate()** method calls it for each object it generates. The method is called after assigning values to the properties but before the object is saved to disk. Each call to the **Populate()** method results in a check for the existence of the **OnPopulate()** method and a call to **OnPopulate()** it for each object it generates.

This instance method is called by the **Populate** method after assigning values to properties but before the object is saved to disk. This method provides additional control over the generated data. If an **OnPopulate()** method exists, then the **Populate** method calls it for each object that it generates.

Its signature is:

### Class Member

```
Method OnPopulate() As %Status
{
    // body of method here...
}
```

**Note:** This is not a private method.

The method returns a %Status code, where a failure status causes the instance being populated to be discarded.

For example, if you have a stream property, Memo, and wish to assign a value to it when populating, you can provide an **OnPopulate()** method:

### Class Member

```
Method OnPopulate() As %Status
{
    Do ..Memo.Write("Default value")
    QUIT $$$OK
}
```

You can override this method in subclasses of %Library.Populate.

## 7 Alternative Approach: Creating a Utility Method

There is another way to use the methods of the %Populate and %PopulateUtils classes. Rather than using %Populate as a superclass, write a utility method that generates data for your classes.

In this code, for each class, iterate a desired number of times. In each iteration:

1. Create a new object.
2. Set each property using a suitable random (or nearly random) value.  
To generate data for a property, call a method of %Populate or %PopulateUtils or use your own method.
3. Save the object.

As with the standard approach, it is necessary to generate data for independent classes before generating it for the dependent classes.

## 7.1 Tips for Building Structure into the Data

In some cases, you might want to include certain values for only a percentage of the cases. You can use the **\$RANDOM** function to do this. For example, use this function to define a method that returns true or false randomly, depending on a cutoff percentage that you provide as an argument. So, for example, it can return true 10% of the time or 75% of the time.

When you generate data for a property, you can use this method to determine whether or not to assign a value:

### ObjectScript

```
If ..RandomTrue(15) {  
    set ..property="something"  
}
```

In the example shown here, approximately 15 percent of the records will have the given value for this property.

In other cases, you might need to simulate a distribution. To do so, set up and use a lottery system. For example, suppose that 1/4 of the values should be A, 1/4 of the values should be B, and 1/2 the values should be C. The logic for the lottery can go like this:

1. Choose an integer from 1 to 100, inclusive.
2. If the number is less than 25, return value A.
3. If the number is between 25 and 49, inclusive, return value B.
4. Otherwise, return value C.

